

Cápsula interactiva

IA en React – Buenas Prácticas y Seguridad en SPA.

Introducción

Imagina que tu equipo debe desarrollar una aplicación web SPA (Single Page Application) con React para un cliente que exige seguridad, accesibilidad y rendimiento. No basta con que la aplicación funcione: debe cumplir estándares de calidad y ser mantenible. Aquí entra la Inteligencia Artificial como aliada estratégica: GitHub Copilot acelera la generación de código y SonarQube analiza vulnerabilidades y calidad. Sin embargo, la IA no reemplaza tu criterio técnico: debes saber cómo guiarla con prompts claros, validar cada sugerencia y aplicar buenas prácticas.

En esta cápsula aprenderás:

- Cómo aplicar buenas prácticas y seguridad en SPA con React.
- Cómo usar Copilot y SonarQube para mejorar tu código.
- Cómo aplicar un checklist de calidad y accesibilidad.
- Cómo documentar el uso de IA con una plantilla Anexo IA.

Prepárate para desarrollar aplicaciones más robustas, seguras y accesibles.



Fuente: <https://sl.bing.net/cHzWp4wP7Uy>

Guía de Buenas Prácticas y Seguridad en SPA

En el desarrollo de aplicaciones Single Page Application (SPA) con React, la calidad del código y la seguridad son factores críticos para garantizar un producto confiable, escalable y accesible. Una SPA concentra gran parte de la lógica en el cliente, lo que la hace más vulnerable a errores de implementación y ataques como XSS (Cross-Site Scripting) si no se aplican las medidas adecuadas. Por ello, es fundamental adoptar buenas prácticas de programación, que no solo optimicen el rendimiento y la mantenibilidad, sino que también fortalezcan la protección de datos y la experiencia del usuario.

Esta guía te ayudará a comprender cómo estructurar componentes de manera eficiente, gestionar el estado de forma segura y aplicar principios que reduzcan riesgos en tu aplicación. Además, aprenderás a integrar herramientas de Inteligencia Artificial (IA) como GitHub Copilot para acelerar el desarrollo, sin perder el control sobre la calidad y la seguridad del código. El objetivo es que cada línea que escribas contribuya a una SPA robusta, accesible y alineada con los estándares modernos.

1. Buenas prácticas en React SPA

- Componentización clara: Divide la interfaz en componentes reutilizables.
- Estado seguro: Usa `useState` y `useEffect` evitando mutaciones directas.
- Evita vulnerabilidades comunes:
 - No uses `dangerouslySetInnerHTML` sin sanitización.
 - Valida datos antes de renderizarlos.

Ejemplo básico de componente seguro:

```
src > components > 📁 UserCard.jsx > ...
1  import React from "react";
2
3  /**
4   * Componente seguro que evita XSS al escapar < y >
5   */
6  function UserCard({ name }) {
7      const safeName = String(name)
8          .replace(/</g, "&lt;");
9          .replace(/>/g, "&gt;");
10
11      return <div className="user-card">{safeName}</div>;
12  }
13
14  export default UserCard;
15
```

Figura 2. Código JSX en VS Code – Componente React. Fuente: Elaboración propia (2026).

Buenas prácticas adicionales:

- Prompt: *"Agrega prop-types para validar propiedades"*.

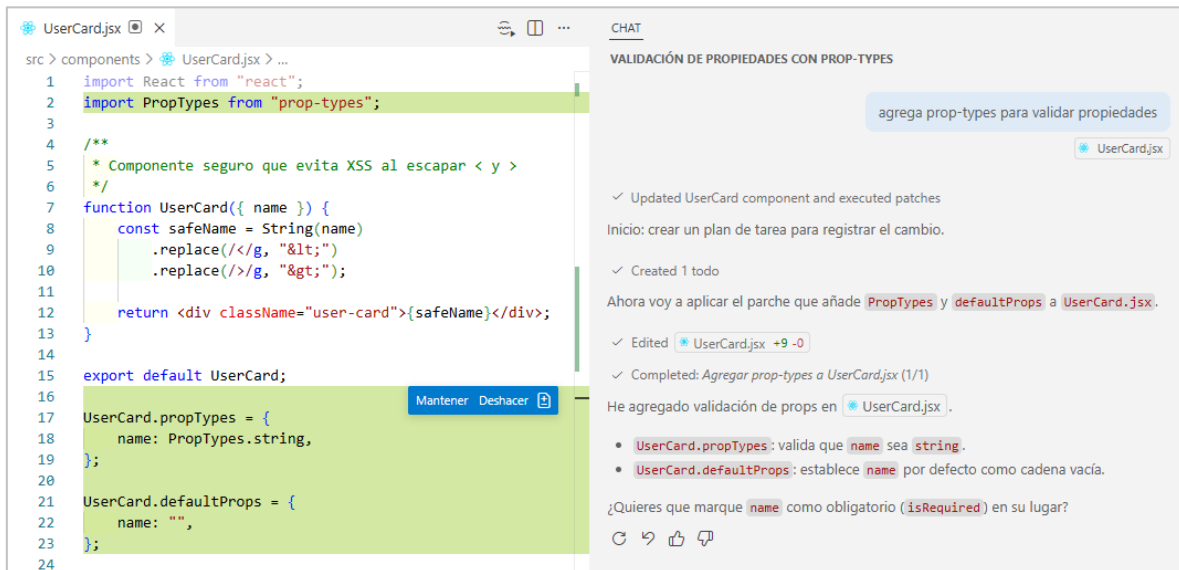


Figura 3. Propuesta de IA: prop-types para validación de propiedades. Fuente: Elaboración propia (2026).

- Prompt: *"Implementa error boundaries para manejar errores en componentes"*.

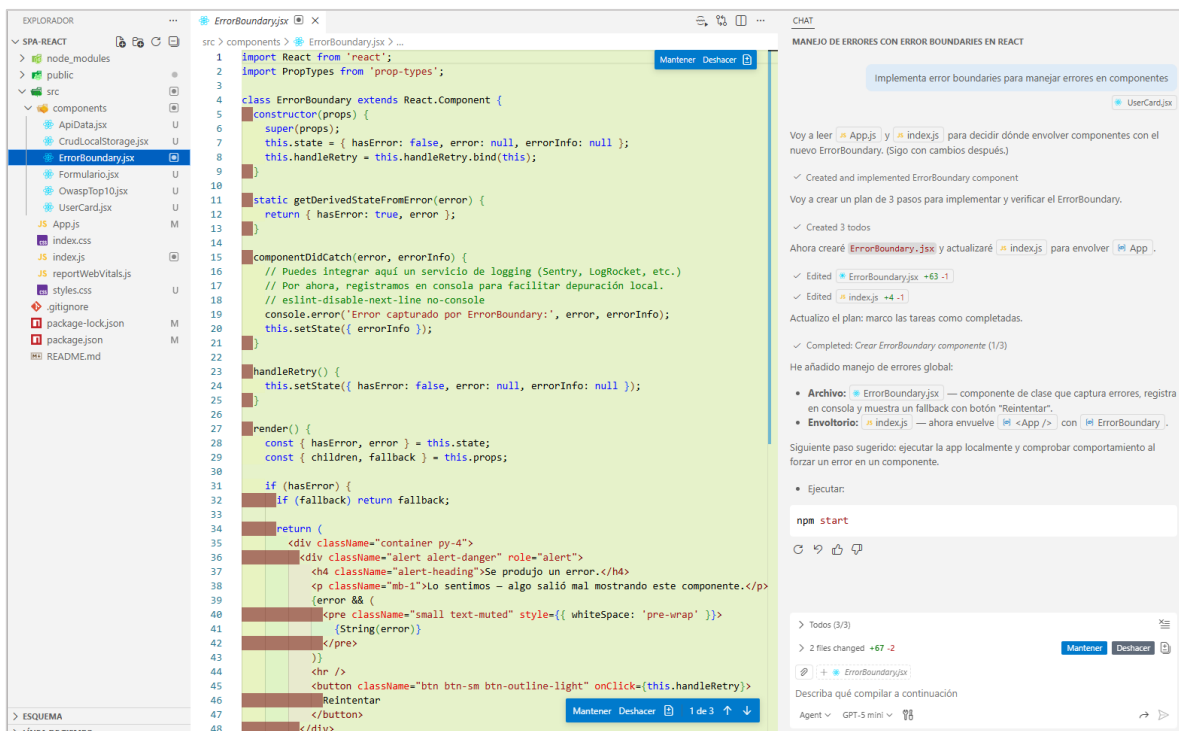


Figura 4. Propuesta de IA: ErrorBoundaries para manejo de errores. Fuente: Elaboración propia (2026).

2. Seguridad y análisis con SonarQube

En el desarrollo de aplicaciones SPA, la seguridad y la calidad del código son esenciales para prevenir vulnerabilidades y mantener la mantenibilidad del proyecto. Aquí es donde herramientas como SonarQube y su extensión SonarLint se convierten en aliados estratégicos: permiten analizar el código en tiempo real, detectar errores comunes, vulnerabilidades y "code smells" antes de que lleguen a producción. Integrar este análisis en tu flujo de trabajo ayuda a cumplir estándares OWASP y aplicar buenas prácticas de programación segura.

- **Instalación:** Agrega la extensión SonarLint en VS Code.
- **Uso:** Analiza el código y corrige vulnerabilidades (XSS, uso inseguro de APIs).
- **Métricas clave:** Code Smells, Vulnerabilities, Security Hotspots.

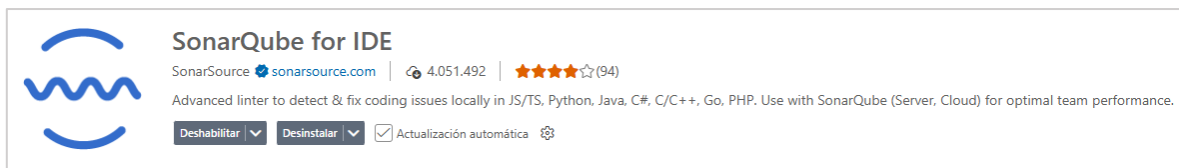


Figura 5. Extensión para VS Code SonarQube for IDE. Fuente: Elaboración propia (2026).

Al activar el modo automático de análisis con SonarQube visualizaras las alertas con el icono de ampolleta amarilla que permitirá corregir o entregar explicaciones de la alerta, mejorando la seguridad del código, como lo muestra la figura 6:

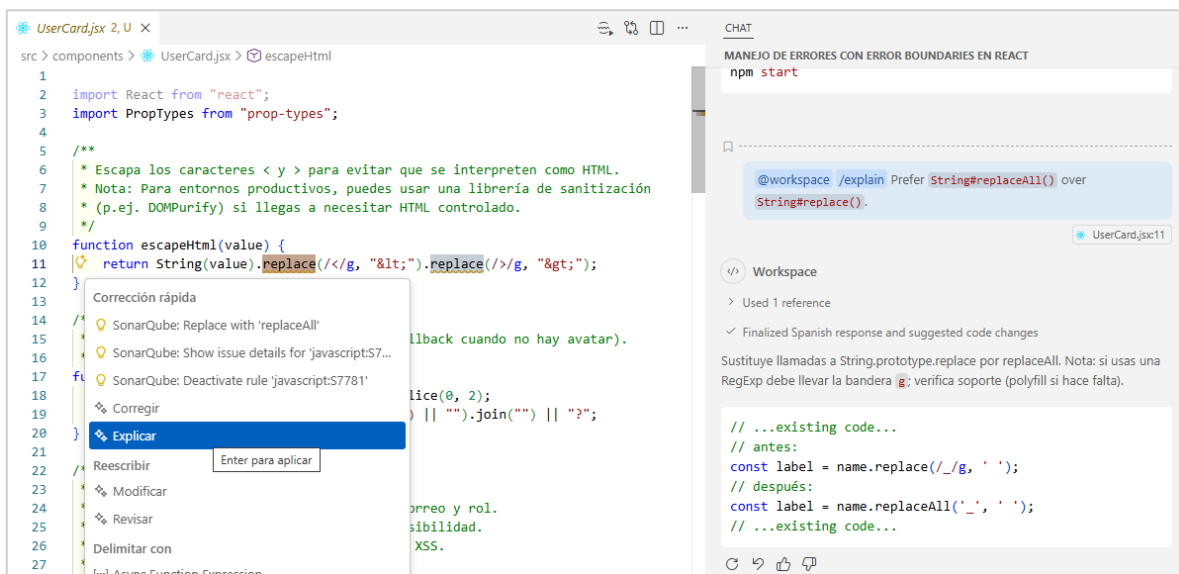


Figura 6. Explicaciones entregadas por SonarQube. Fuente: Elaboración propia (2026).

Resultado:

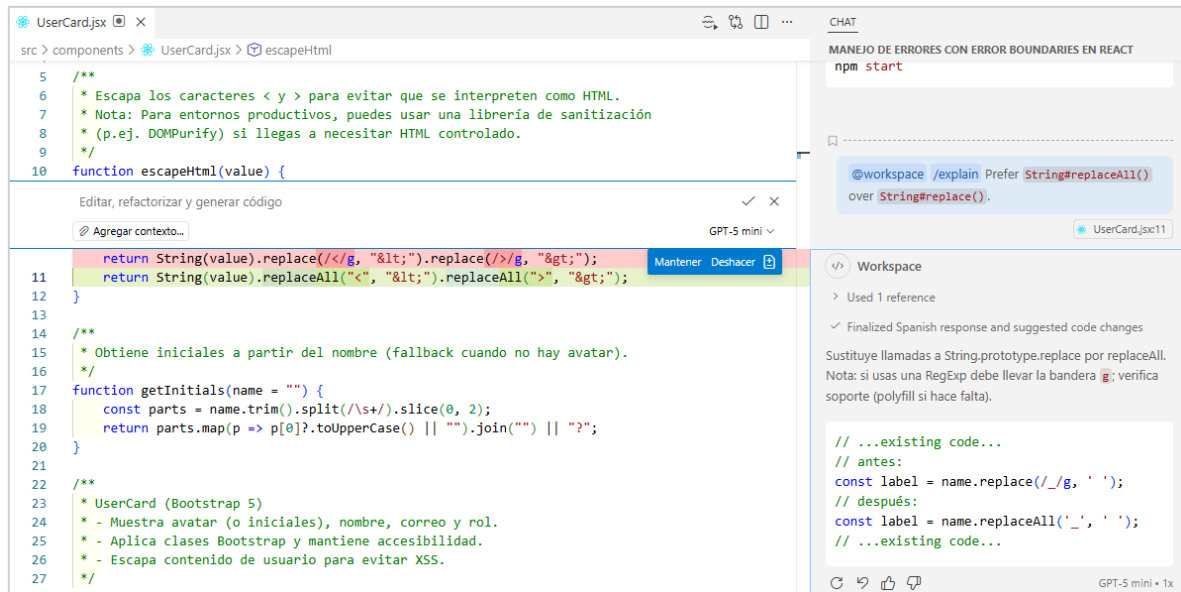


Figura 7. Correcciones sugeridas por SonarQube. Fuente: Elaboración propia (2026).

3. IA como apoyo (Copilot)

La inteligencia artificial no reemplaza tu criterio: lo potencia. En React, GitHub Copilot acelera la escritura y refactorización de componentes, sugiere patrones seguros (p. ej., manejo de errores y accesibilidad) y ayuda a documentar decisiones técnicas sin perder el control humano. La clave está en redactar buenos prompts, revisar críticamente cada sugerencia y contrastar con herramientas de calidad/seguridad como SonarQube antes de integrar el código a la SPA.

3.1. Generación de componentes accesibles

Crear componentes accesibles significa diseñar y codificar desde el inicio pensando en todas las personas, incluidas quienes navegan con teclado o lector de pantalla. Con Copilot puedes acelerar la construcción de formularios y vistas, pero tu rol es guiarlo con prompts que exijan semántica correcta (labels asociados, encabezados jerárquicos), **retroalimentación accesible** (roles [alert/status](#), [aria-describedby](#)), **foco visible** y **operación por teclado**. La combinación de buenas prácticas A11y + sugerencias de IA reduce barreras, evita retrabajos y mejora la calidad percibida, manteniendo siempre la **revisión crítica humana** antes de integrar el código a la SPA.

Prompt:

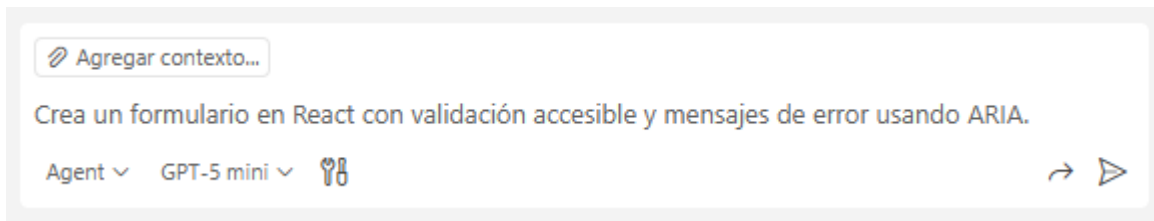


Figura 8. Prompt Chat GitHub Copilot – Solicitud de formulario en React. Fuente: Elaboración propia (2026).

Propuesta:

Copilot advierte que ya tenemos un formulario existente y propone actualizarlo para dar cobertura a la solicitud generada a través del Prompt, como lo indica la figura 9:

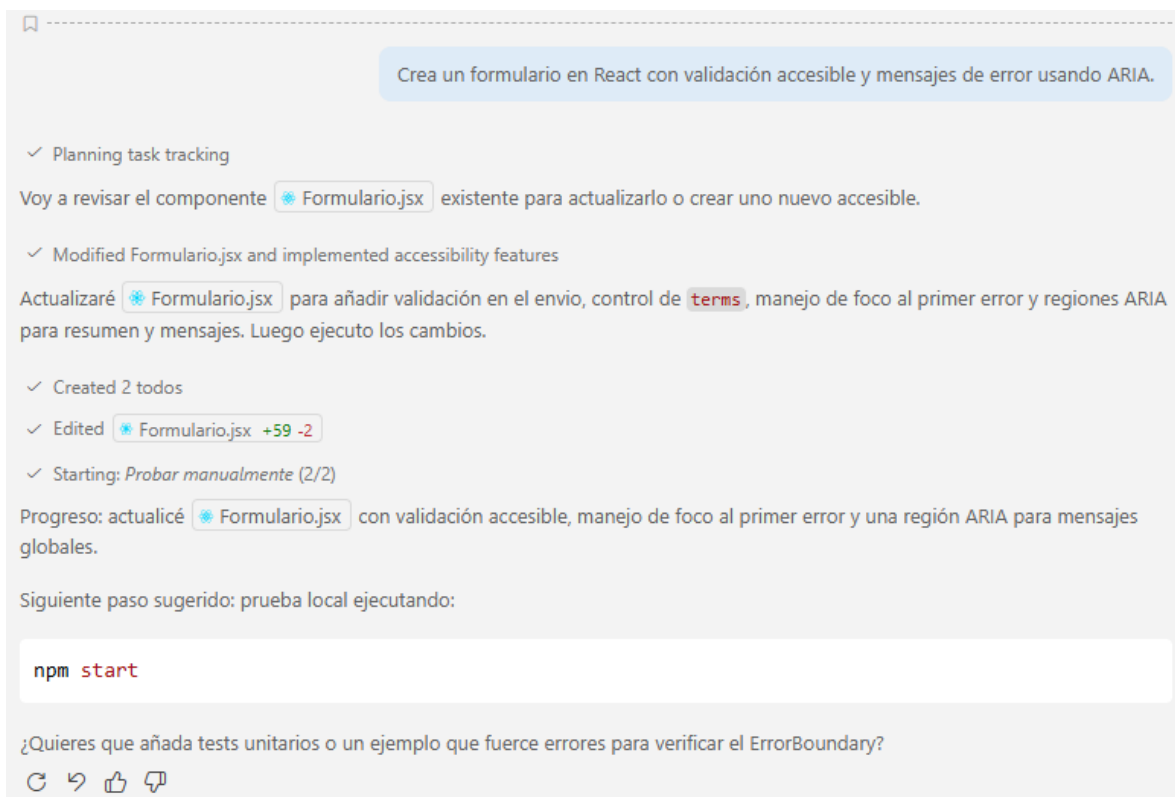


Figura 9. Sugerencias entregadas por Chat GitHub Copilot – Actualización de formulario existente. Fuente: Elaboración propia (2026).

Resultado:

Mensaje de error del campo email:

```
    {/* Mensaje de error accesible */}
    {isInvalid && (
      <div id="email-feedback" className="invalid-feedback" role="alert">
        {error}
      </div>
    )}
```

Figura 10. Código JSX en VS Code – Mensaje de error accesible creado por Copilot. Fuente: Elaboración propia (2026).

Región global de estado – Mensajes de formulario:

```
    {/* Mensaje global de estado (errores de formulario o éxito) */}
    <div aria-live="assertive" aria-atomic="true" className="mt-3">
      {formMessage && (
        <div className={`alert ${error ? 'alert-danger' : 'alert-success'}`} role="alert">
          {formMessage}
        </div>
      )}
    </div>
```

Figura 11. Código JSX en VS Code – Mensaje de error con aria-live creado por Copilot. Fuente: Elaboración propia (2026).

3.2. CRUD con LocalStorage

Un CRUD (Create, Read, Update, Delete) con LocalStorage permite gestionar datos en el navegador sin depender de un servidor, ideal para prácticas iniciales en SPA. Con Copilot puedes generar rápidamente la estructura básica, pero tu rol es asegurar que el código cumpla buenas prácticas: validar entradas, evitar duplicados, manejar errores y mantener la persistencia de forma segura. Este enfoque enseña cómo organizar componentes, usar hooks como useState y useEffect, y aplicar principios de accesibilidad en formularios y mensajes, reforzando la lógica de negocio y la experiencia del usuario.

Prompt:

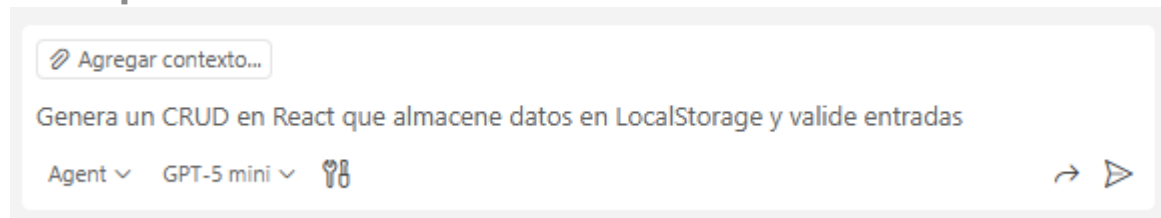


Figura 12. Prompt Chat GitHub Copilot – Solicitud de CRUD en React que almacene en LocalStorage. Fuente: Elaboración propia (2026).

Propuesta:

Copilot indica que creara un componente CRUD mejorado que persiste en LocalStorage y valida entradas, como lo indica la figura 13:

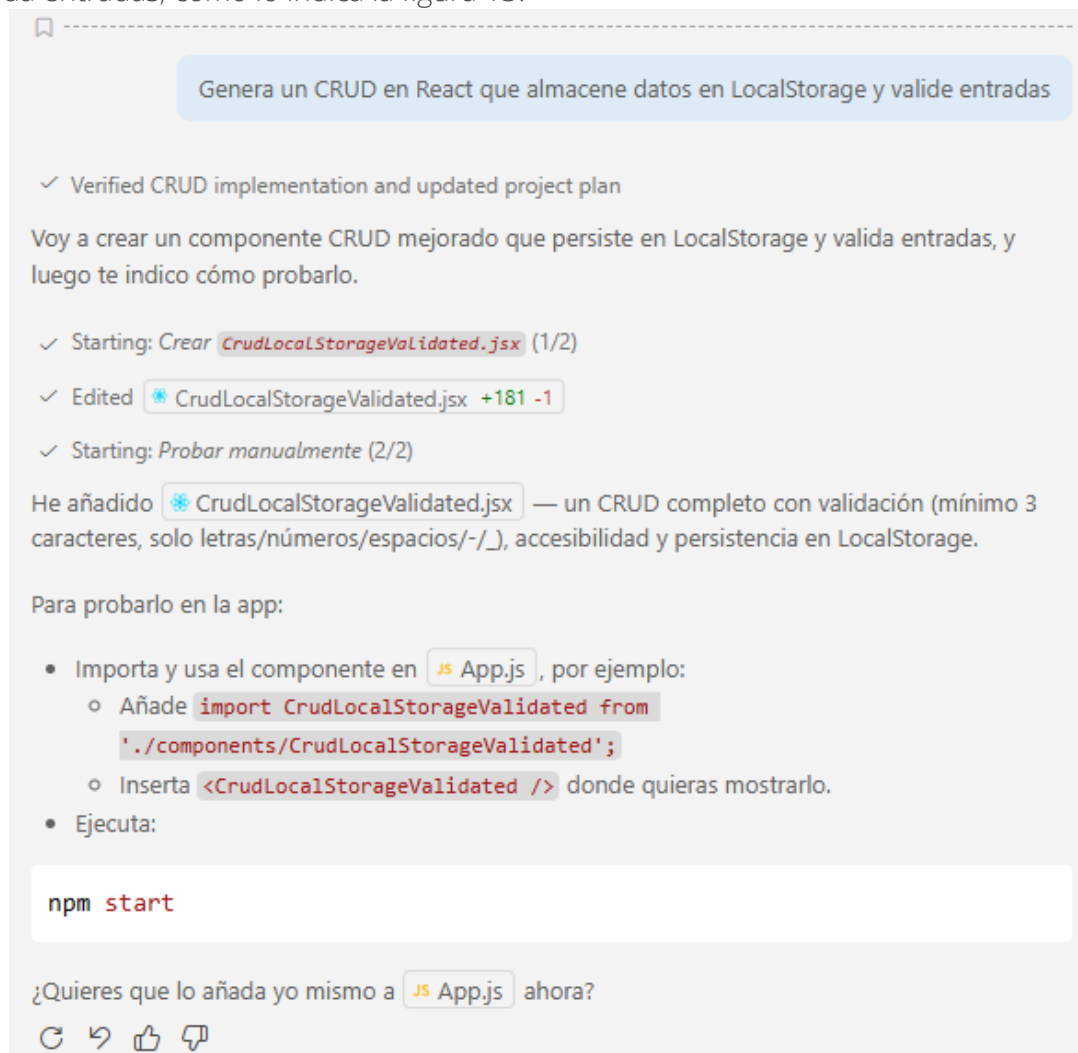


Figura 13. Sugerencias entregadas por Chat GitHub Copilot – Generación de CRUD en LocalStorage para React. Fuente: Elaboración propia (2026).

Resultado:

Archivo CrudLocalStorageValidated.jsx creado a través de Chat GitHub Copilot:

```

CrudLocalStorageValidated.jsx 5, U X
src > components > CrudLocalStorageValidated.jsx > CrudLocalStorageValidated
1  import React, { useEffect, useRef, useState } from 'react';
2
3  /**
4   * CRUD con validación más estricta y persistencia en LocalStorage.
5   * - Reglas: mínimo 3 caracteres, solo letras, números, espacios, guión y guion bajo.
6   * - Operaciones: crear, leer, actualizar, eliminar, vaciar.
7   * - Accesibilidad: `role="alert"`, `aria-live`, focus en error.
8   */
9  const KEY = 'items_validated_v1';
10
11  function isValidValue(v) {
12    if (!v) return false;
13    const s = v.trim();
14    if (s.length < 3) return false;
15    // permitidos: letras, números, espacios, -, _
16    return /^[A-Za-z0-9 _-]+$/.test(s);
17  }
18
19  export default function CrudLocalStorageValidated() {
20    const [items, setItems] = useState([]);
21    const [value, setValue] = useState('');
22    const [editing, setEditing] = useState(null);
23    const [filter, setFilter] = useState('');
24    const [message, setMessage] = useState({ type: '', text: '' });
25
26    const inputRef = useRef(null);
27
28    useEffect(() => {
29      try {
30        const raw = localStorage.getItem(KEY);
31        setItems(raw ? JSON.parse(raw) : []);
32      } catch (e) {

```

Figura 14. Código JSX en VS Code – Fragmento inicial del código para CrudLocalStorageValidated creado por Copilot. Fuente: Elaboración propia (2026).

3.3. Consumo de API con manejo de errores

Consumir una API no es solo “pegarle” a un endpoint y parsear la respuesta. En entornos reales —y especialmente en sistemas críticos o con exposición a Internet— el **manejo de errores** es un pilar de **confiabilidad, seguridad y experiencia de usuario**. Una integración robusta **anticipa fallos, los clasifica, responde proporcionalmente y se recupera** sin comprometer la integridad de los datos ni filtrar información sensible.

Prompt:

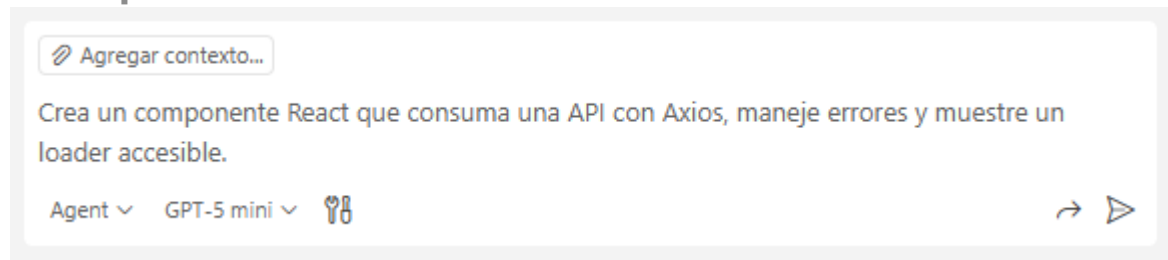


Figura 15. Prompt Chat GitHub Copilot – Solicitud Componente React para consumo de API con Axios. Fuente: Elaboración propia (2026).

Propuesta:

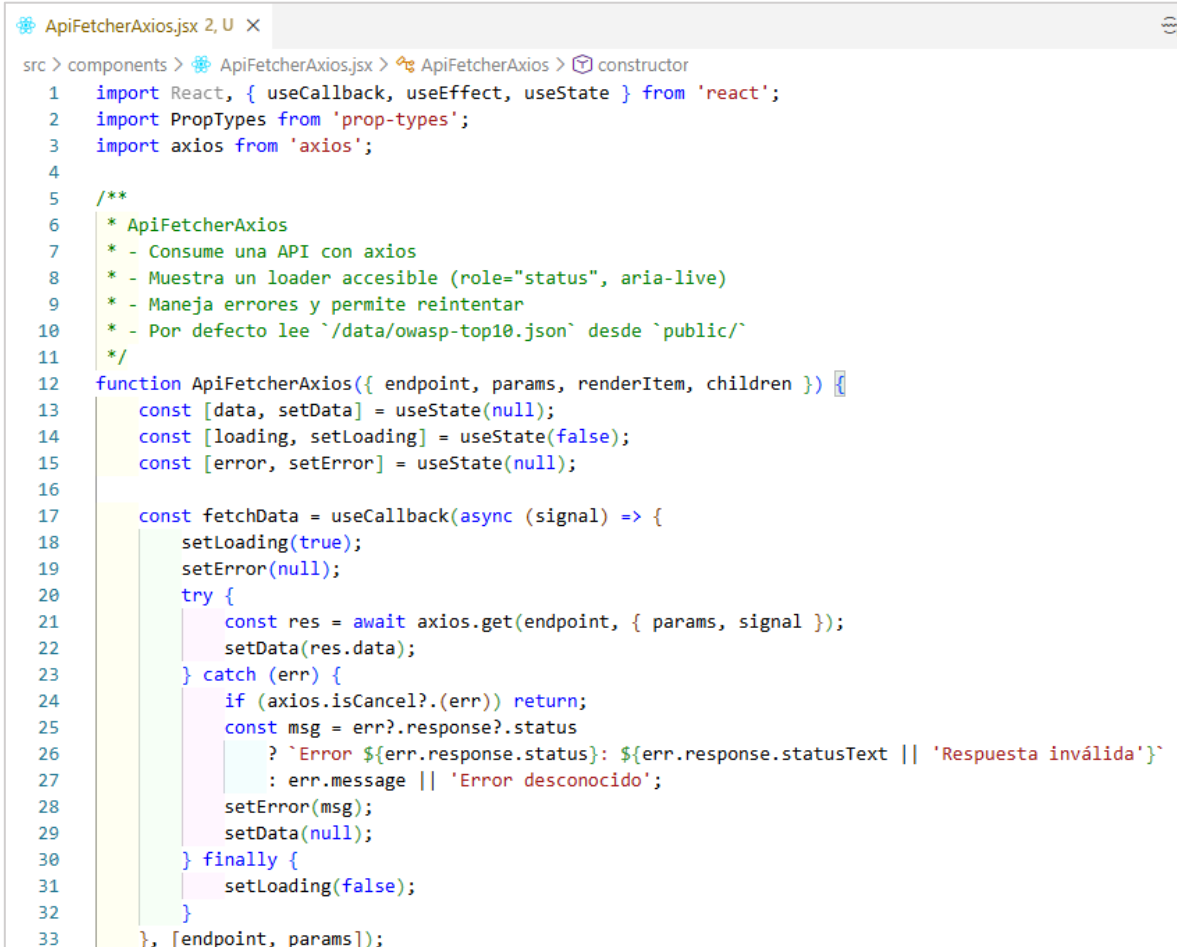
Copilot indica que creara un componente CRUD mejorado que persiste en LocalStorage y valida entradas, como lo indica la figura 13:



Figura 16. Sugerencias entregadas por Chat GitHub Copilot – Generación de Componente React para consumo de API con Axios. Fuente: Elaboración propia (2026).

Resultado:

Archivo ApiFetcherAxios.jsx creado a través de Chat GitHub Copilot:



```
src > components > ApiFetcherAxios.jsx > ApiFetcherAxios > constructor
1  import React, { useCallback, useEffect, useState } from 'react';
2  import PropTypes from 'prop-types';
3  import axios from 'axios';
4
5  /**
6   * ApiFetcherAxios
7   * - Consume una API con axios
8   * - Muestra un loader accesible (role="status", aria-live)
9   * - Maneja errores y permite reintentar
10  * - Por defecto lee `/data/owasp-top10.json` desde `public/`
11  */
12  function ApiFetcherAxios({ endpoint, params, renderItem, children }) {
13    const [data, setData] = useState(null);
14    const [loading, setLoading] = useState(false);
15    const [error, setError] = useState(null);
16
17    const fetchData = useCallback(async (signal) => {
18      setLoading(true);
19      setError(null);
20      try {
21        const res = await axios.get(endpoint, { params, signal });
22        setData(res.data);
23      } catch (err) {
24        if (axios.isCancel?.(err)) return;
25        const msg = err?.response?.status
26          ? `Error ${err.response.status}: ${err.response.statusText || 'Respuesta inválida'}`
27          : err.message || 'Error desconocido';
28        setError(msg);
29        setData(null);
30      } finally {
31        setLoading(false);
32      }
33    }, [endpoint, params]);
```

Figura 17. Código JSX en VS Code – Fragmento inicial del código creado para ApiFetcherAxios por Copilot.
Fuente: Elaboración propia (2026).

4. Checklist de Calidad y Accesibilidad (A11y) con apoyo de IA

La calidad y la accesibilidad son pilares para aplicaciones inclusivas y confiables. Hoy, la Inteligencia Artificial (IA) potencia estas tareas, automatizando auditorías, detectando problemas y sugiriendo mejoras en tiempo real.

¿Por qué usar IA en A11y?

- **Automatización inteligente:** IA analiza interfaces y código para detectar errores de accesibilidad (contraste, etiquetas, roles ARIA) sin intervención manual.
- **Corrección asistida:** Genera recomendaciones y fragmentos de código accesible (ej.: añadir alt adecuado, roles correctos).
- **Aprendizaje continuo:** Modelos entrenados con estándares WCAG y patrones UX sugieren mejoras adaptadas al contexto.
- **Escalabilidad:** Auditar cientos de páginas o componentes en segundos, reduciendo costos y tiempo.

Elementos del Checklist potenciados por IA

1. **Detección automática de problemas:**
 - Herramientas IA (ej.: **axe-core con IA**, **Microsoft Accessibility Insights**, **Lighthouse con IA**) identifican errores semánticos, falta de etiquetas, bajo contraste.
2. **Generación de soluciones:**
 - IA propone código corregido:
Ejemplo: "Falta alt en imagen" → IA sugiere alt="Gráfico de ventas 2024" basado en contexto.
3. **Validación predictiva:**
 - IA predice impacto en usuarios con discapacidad y prioriza correcciones críticas.
4. **Pruebas automatizadas con IA:**
 - Simulación de navegación por teclado y lectores de pantalla usando bots inteligentes.

5. Monitoreo continuo:

- IA integrada en CI/CD alerta sobre regresiones en accesibilidad antes del despliegue.

5. Plantilla Anexo IA

La inclusión de una **Plantilla Anexo IA** en el documento responde a la necesidad de **estandarizar el uso de Inteligencia Artificial (IA)** en procesos académicos y profesionales, garantizando **transparencia, ética y trazabilidad**. Este anexo permite documentar cómo, cuándo y con qué herramientas se aplicó IA, evitando riesgos asociados a plagio, sesgos y falta de control en la generación de contenido.

Razones principales

1. Transparencia y trazabilidad:

- Documentar el uso de IA asegura que los evaluadores y auditores comprendan el alcance y propósito de la asistencia tecnológica.
- Permite diferenciar entre aportes humanos y sugerencias generadas por IA.

2. Cumplimiento normativo y ético:

- Cada vez más instituciones exigen declarar el uso de IA para cumplir con políticas de integridad académica y estándares internacionales (IEEE, UNESCO).
- Evita sanciones por uso indebido o falta de atribución.

3. Mejora de calidad y reproducibilidad:

- Una plantilla estandarizada facilita replicar procesos, evaluar resultados y garantizar consistencia en proyectos futuros.

4. Fomento de buenas prácticas:

- Promueve el uso responsable de IA, incluyendo verificación humana y referencias a fuentes confiables Herramientas

Formato sugerido para la Plantilla Anexo IA

Título:

Anexo IA – Declaración de Uso de Inteligencia Artificial.

Secciones:

- 1. **Herramienta utilizada:** (Ej.: Microsoft Copilot, ChatGPT, MidJourney)
- 2. **Propósito del uso:** (Ej.: Generación de código, redacción de texto, análisis de datos)
- 3. **Fecha y contexto:** (Ej.: Unidad 3.3 – Consumo de API con manejo de errores)
- 4. **Intervención humana:** (Ej.: Revisión, validación, ajustes manuales)
- 5. **Fuentes complementarias:** (Ej.: Documentación oficial, artículos académicos)
- 6. **Declaración ética:**

"El contenido generado por IA fue revisado y validado por el autor, asegurando su pertinencia y exactitud."

Ejemplo visual:

Campo	Detalle
Herramienta IA	Microsoft Copilot
Propósito	Generación de ejemplos de código y guía
Fecha	15/01/2026
Intervención humana	Ajuste de sintaxis y validación funcional
Fuentes complementarias	WCAG 2.1, W3C, Microsoft Docs

Tabla 1. Propuesta de IA: Formato sugerido Anexo IA. Fuente: Elaboración propia (2026).

Fuentes en formato APA

- World Wide Web Consortium (W3C). (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. Recuperado de <https://www.w3.org/TR/WCAG21/>
- UNESCO. (2023). *Recomendación sobre la ética de la inteligencia artificial*. París: UNESCO.
- IEEE Standards Association. (2022). *Ethically Aligned Design: A Vision for Prioritizing Human Well-being with Autonomous and Intelligent Systems*. IEEE.

Recursos complementarios

A continuación, encontrarás recursos para complementar lo aprendido sobre las IA en React – Buenas Prácticas y Seguridad en SPA

Brenet, David. (2024). *La inteligencia artificial explicada: De los conceptos básicos a las aplicaciones avanzadas de IA*. Recuperado de URL:

https://inacap.primo.exlibrisgroup.com/permalink/56INACAP_INST/it4brd/alma991000702047406726

Firdaus, Thoriq, Ben Frain, and Benjamin LaGrone. HTML5 and CSS3 : Building Responsive Websites : Design Robust, Powerful, and above All, Modern Websites across All Manner of Devices with Ease Using HTML5 and CSS3 : A Course in Three Modules. 1st edition. Birmingham, England: Packt Publishing, 2016. Print.

Recuperado de URL:

https://inacap.primo.exlibrisgroup.com/permalink/56INACAP_INST/it4brd/alma991000617648506726

Referencias bibliográficas

Fregly, Chris, Antje Barth, y Shelbee Eigenbrode. *IA Generativa en AWS : Creación de Aplicaciones Multimodales*. 1st ed. Barcelona: Marcombo, S.A., 2024. Print.

https://inacap.primo.exlibrisgroup.com/permalink/56INACAP_INST/it4brd/alma991000725515006726

Actividad formativa

La siguiente actividad es para que practiques y midas tu aprendizaje, referente al contenido revisado en este recurso. Puedes realizarla las veces que desees.

Instrucciones:

Lee cada una de las preguntas y selecciona la alternativa que consideres correcta.

¡Mucho éxito!

1. ¿Cuál es la mejor práctica para evitar vulnerabilidades XSS en una SPA desarrollada con React?

@ Validar datos antes de renderizarlos y evitar el uso de dangerouslySetInnerHTML sin sanitización. Esto reduce el riesgo de ejecución de código malicioso.

- a) Usar dangerouslySetInnerHTML para insertar contenido dinámico.
- b) *Validar datos y evitar dangerouslySetInnerHTML sin sanitización.**
- c) Renderizar cualquier entrada del usuario directamente en el DOM.
- d) Deshabilitar el uso de hooks para prevenir ataques.

2. ¿Qué herramienta permite analizar vulnerabilidades y calidad del código en tiempo real en React?

@ SonarQube (y su extensión SonarLint) detecta errores, vulnerabilidades y “code smells” antes de llegar a producción, ayudando a cumplir estándares OWASP.

- a) *SonarQube / SonarLint.**
- b) ESLint.
- c) Prettier.
- d) Webpack.

3. ¿Cuál es el rol principal de GitHub Copilot en el desarrollo de una SPA?

@ Copilot acelera la generación y refactorización de código, sugiere patrones seguros y accesibles, pero requiere revisión crítica humana antes de integrar.

- a) Sustituir completamente la revisión humana.
- b) Ejecutar pruebas automatizadas sin intervención.
- c) Garantizar la seguridad sin necesidad de validación.

d) *Sugerir código y patrones seguros, bajo supervisión humana.

4. ¿Qué elemento es clave para garantizar accesibilidad (A11y) en componentes React?

@ Incluir roles ARIA, etiquetas semánticas y retroalimentación accesible (ej.: aria-live) mejora la experiencia para usuarios con discapacidad.

- a) Ignorar atributos ARIA para simplificar el código.
- b) *Usar roles ARIA, etiquetas correctas y retroalimentación accesible.
- c) Deshabilitar navegación por teclado para evitar errores.
- d) Implementar solo estilos visuales sin semántica.

5. ¿Por qué es importante documentar el uso de IA mediante la Plantilla Anexo IA?

@ Garantiza transparencia, ética y trazabilidad, diferenciando aportes humanos de sugerencias IA y cumpliendo estándares internacionales.

- a) Para ocultar el uso de IA en el proyecto.
- b) Para reducir el tiempo de desarrollo sin controles.
- c) *Para asegurar transparencia, trazabilidad y cumplimiento normativo.
- d) Para evitar la revisión crítica del código generado.

Cierre

Llegar hasta aquí significa que diste un paso importante hacia un desarrollo más consciente y profesional. Crear una SPA en React no se trata solo de que “funcione”, sino de que sea segura, accesible y fácil de mantener. Hoy aprendiste que detrás de cada decisión técnica hay un impacto real: proteger datos, evitar vulnerabilidades y ofrecer una experiencia inclusiva para todas las personas. Las herramientas de IA como GitHub Copilot y SonarQube son grandes aliados, pero no sustituyen tu criterio; al contrario, lo potencian cuando las usas con responsabilidad y sentido crítico. Documentar su uso, aplicar buenas prácticas y pensar en accesibilidad desde el inicio son señales de madurez profesional. Recuerda que la calidad no es un lujo, es una obligación ética y técnica. Cada línea de código que escribas puede marcar la diferencia entre una aplicación confiable y una llena de riesgos. Sigue explorando, probando y mejorando: el verdadero valor está en iterar con propósito. Así, tu trabajo no solo será funcional, sino que reflejará compromiso, seguridad y respeto por quienes usarán tu aplicación.